# AI-DLC Pipeline — Optimization Ideas


- **Author:** Varshil Gandhi (varshil.g@codiste.in)

- **Date:** 2026-05-07
- **Context:** After running the e-commerce MVP through both SDLC (vibe coding) and AI-DLC end-to-end, these are my ideas on how we can optimize the AI-DLC pipeline to fit Codiste — across projects, roles, and tooling. This is the raw input document that fed the broader *Codiste-AIDLC Optimization Plan*; the ideas here are intentionally kept as proposals, not detailed implementations.

---

## Background

I built the same e-commerce MVP twice — once with SDLC vibe coding, once with the full AI-DLC pipeline. The AI-DLC version delivered a working system with a complete audit trail (5 gates, 12 UoWs, ~1,158 tests passing), but I also saw places where the pipeline could be tighter, faster, and more useful for the way Codiste actually works. Below are the four ideas I want to put on the table, plus a tooling baseline that should ship with the pipeline by default.

---

## Idea 1 — Pre-Inception project-type wizard

**Proposal.** Before any AI-DLC stage runs, the pipeline should ask one structured question:

> *"What kind of project is this? — MVP / Greenfield / Brownfield extension / Brownfield modernization / Spike."*

The answer routes the rest of the pipeline:

- **MVP** → trimmed pipeline focused on speed and shippability
- **Greenfield** → full AI-DLC (what we just ran)
- **Brownfield extension** → "AI-DLC Brownfield Lite" — adds a Discovery stage, drops Stack Selection, focuses on integration with existing code
- **Brownfield modernization** → full AI-DLC + a mandatory Cutover stage
- **Spike** → skip AI-DLC entirely, vibe-code on a throwaway branch

**Why this matters.** Right now the same heavy pipeline runs for every project. A 50-line bug fix doesn't need 28k lines of process docs; a regulated FinTech rebuild needs every stage. Asking the question up front — once, in 30 seconds — replaces a lot of mid-flow judgment calls and keeps the audit trail consistent.

---

## Idea 2 — GitHub-native pipeline (UoW = branch, Gate = PR)

**Proposal.** Replace markdown signatures with GitHub-native sign-offs by making three things automatic:

1. **At project start:** AI-DLC offers to set up the GitHub repo and the GitHub MCP server (private repo by default, MCP installed for the session).
2. **Per UoW:** when a new Unit of Work begins, the pipeline automatically creates a feature branch ( `uow/NN-<slug>` ) and lands all Construction commits there.
3. **At sign-off:** developer pushes to the feature branch and opens a PR. Manager review on the PR = sign-off. Manager approve + merge = gate sealed. No manual markdown editing of signature blocks.

**Why this matters.** Right now sign-offs are markdown signatures inside docs, manually written by both pod members. That's hard to audit, easy to fake (a name typed into a file), and disconnected from the actual code change. GitHub PR review is the natural fit for the pod model — every gate becomes a real two-person review with immutable history. For Codiste's regulated-vertical clients (FinTech, RegTech, HIPAA), this *is* the audit trail.

---

## Idea 3 — Role-based optimization (tech + non-tech)

**Proposal.** Pre-Inception, ask one more structured question:

> *"What's your role? — Founder / Product Manager / BDE / Tech Lead / Frontend Dev / Backend Dev / QA / DevOps."*

Pipeline adapts per role:

- **BDE** → idea brief → discovery questions → competitive scan → **client pitch deck + 1-page proposal** as the deliverable (not code)
- **Product Manager** → idea → user research → **PRD + roadmap + success metrics** (handed off to Tech track)
- **Tech Lead** / **Full-stack Dev** → full AI-DLC (current behavior)
- **Frontend Dev** → design intake → component spec → implementation → visual regression (frontend-only stages)
- **Backend Dev** → data model → API design → implementation → contract tests (backend-only stages)
- **QA** → test plan → test cases → automation suite (test artifacts traceable to UoWs)
- **DevOps** → IaC scaffold → deployment plan → observability → runbook

**Why this matters.** AI-DLC right now assumes a developer audience. At Codiste, BDEs and Product people have idea-to-output workflows that look very similar — *idea → understand it → produce a structured artifact → get sign-off* — but their artifact isn't code, it's a pitch or a PRD. If we extend the

pipeline to cover them with the same gate model, the whole company runs on one ledger. A BDE's pitch deck and a backend dev's API design carry the same sign-off format. That's the productization unlock for Codiste — one common methodology across roles, not one for engineers and a different one for sales.

---

## Idea 4 — Use `grill-me` at Inception for shared understanding

**Proposal.** At the start of the Inception phase, run the `grill-me` skill (or an equivalent structured-questioning skill) seeded with the project profile from Idea 1. The skill's job is to grill the user with sharp questions — assumptions, edge cases, constraints, "what would break this" — until a **shared understanding doc** is on the table. That doc gets a Tech Lead sign-off as a new gate (call it Gate #0) before any other Inception artifact is drafted.

**Why this matters.** The single biggest failure mode of Inception in our e-commerce experiment was "Inception looked complete but actually the user hadn't surfaced the constraint that mattered" — which is what produced 9 fix commits *after* Gate #5. `grill-me` forces the structured questioning *before* docs are written, when it's cheap to redirect, instead of after Gate #5 when fixes mean re-opening sealed gates. Thirty minutes of structured Q&A buys multi-day rework savings downstream.

---

## Bonus — Claude Desktop tooling baseline (from the Avthar video I watched)

These shouldn't be optional tips. They should be the **default working environment** every Codiste engineer is set up with on day one — same way we'd standardize an IDE or a linter config. Bundle them into a single Codiste-AIDLC bootstrap.

| # | Convention | What | Codiste rule |
|---|---|---|---|
| 1 | Worktree-per-session checkbox | Multi-Claude is one click — each session gets its own working copy on its own branch | Mandatory for parallel UoWs; one Claude per UoW |
| 2 | Three-column layout (chat / preview / terminal+plan) | Drag-and-drop tile layout snaps panels into place | Default layout for any active build session |
| 3 | Multi-session (up to 4 quadrants) + Claude merges back | Run several sessions, each on its own worktree; Claude merges back to one PR or one test branch | Use only when 2–4 UoWs are *truly* independent — not for dependent work |
| 4 | Inline comments on the Plan, not accept/reject | Plans live in their own pane; comment on specific parts to revise instead of binary accept | Plan revisions must be inline-commented, not re-prompted from scratch |
| 5 | Explicitly request the AskUserQuestion tool | "Use AskUserQuestion while planning" — Claude pauses with structured multiple-choice prompts | Mandatory in Profile Wizard (Idea 1) and `grill-me` Inception (Idea 4) |
| 6 | Cmd+; for side chats | Branch off a side chat that reads main context but doesn't write back | Use for "would-this-work-later" tangents; main session stays clean |
| 7 | Diff viewer with line-level inline comments | GitHub-PR-style review *inside* Claude Code — click any line, leave a comment | Code Review (Gate #4) uses this exclusively — no separate review docs |
| 8 | Sessions grouped by repo in the sidebar | Sidebar groups by repo or recency; sessions auto-archive when their PR closes | Default sidebar setting for all engineers |
| 9 | Cloud sessions for long unattended jobs | Sessions running on Anthropic's infra — keep working with laptop closed | Long Inception research / overnight builds run as cloud sessions |
| 10 | Output modes (Normal / Thinking / Verbose / Summary) | Switch verbosity per task | Verbose for debugging; Summary on session resume after a break |
| 11 | Skills + plugins parity (desktop ↔ CLI) | Whatever works in terminal Claude Code works identically in the desktop app | Codiste skill bundle (`grill-me`, `aidlc-stage-*`, sign-off helpers) installed once, runs everywhere |

**Why this matters.** Most of the friction we hit on the e-commerce project (parallel UoW conflicts, plan re-prompting, manually summarizing what was done so far) is solved by tools that already exist — we just weren't using them as defaults. Standardizing the layout and the bundle eliminates the per-engineer "figure it out" tax.

## Closing note

These four ideas (plus the tooling baseline) are the inputs I think we should design Codiste-AIDLC around. Idea 1 fixes the "one-size pipeline" problem, Idea 2 fixes the "sign-offs aren't real" problem, Idea 3 unlocks non-tech adoption across Codiste, and Idea 4 prevents the post-Gate-#5 fix-commit wave I saw on the e-commerce project. The tooling baseline makes all four cheap to operate.

The detailed integration plan — how these fit together with what AWS AI-DLC already ships, the productization roadmap, the gate flow — is in the separate *Codiste-AIDLC Optimization Plan* document. This file is just the source ideas, captured as-is so the thinking is preserved before it gets folded into the broader plan.